Unit II

2. SQL Statements & Working with Tables

2.1 Introduction to SQL

2.2. SQL Commands and its types (DDL, DML, DQL, DCL, Transaction Control Commands)

2.3 Data types in SQL

2.4 Creating Tables, Selecting from tables, WHERE Clause

2.5 Selecting from tables, DISTINCT Clause, Column aliasing

2.6 Manipulation Table data

2.7 Altering Table structure

2.8 Data Constraints: Unique, Not Null, Primary Key, Foreign Key, Check, Default Constraint

---

2.1 Introduction to SQL:

SQL (Structured Query Language) is a standard programming language used to manage and manipulate relational databases. It allows users to interact with databases to store, retrieve, update, and delete data.

Key Features of SQL:

1. Data Definition Language (DDL):

Commands like `CREATE`, `ALTER`, and `DROP` are used to define and modify database structures.

2. Data Manipulation Language (DML):

Commands like `INSERT`, `UPDATE`, `DELETE`, and `SELECT` are used to manipulate data within tables.

3. Data Querying:

The `SELECT` statement retrieves data from one or more tables using filters and conditions.

4. Data Control Language (DCL):

Commands like `GRANT` and `REVOKE` are used to control access to data.

5. Transaction Control (TCL):

Commands like `COMMIT`, `ROLLBACK`, and `SAVEPOINT` ensure data consistency during transactions.

Advantages of SQL:

Standardized:

Used universally across various relational database systems like MySQL, PostgreSQL, and Oracle.

Powerful and Flexible:

Supports complex queries for data analysis.

User-Friendly:

Easy to learn and use, even for beginners.

SQL is a foundational tool for database management and is widely used in industries for applications like business analytics, e-commerce, and data science.

2.2. SQL Commands and its types (DDL, DML, DQL, DCL, Transaction Control Commands):

SQL commands are categorized based on their purpose in interacting with a relational database.

The main types are:

1. Data Definition Language (DDL):

Used to define and manage the structure of a database.

Commands:

`CREATE`:

To create a database, table, or other objects.

Example:

`CREATE TABLE Students (ID INT, Name VARCHAR(50));`

`ALTER`:

To modify the structure of an existing table.

Example:

`ALTER TABLE Students ADD Age INT;`

`DROP`:

To delete a database or table.

Example:

`DROP TABLE Students;`

2. Data Manipulation Language (DML):

Used to manipulate data stored in tables.

Commands:

`INSERT`:

To add new data.

Example:

`INSERT INTO Students (ID, Name) VALUES (1, 'Ali');`

`UPDATE`:

To modify existing data.

Example:

`UPDATE Students SET Name = 'Ahmed' WHERE ID = 1;`

`DELETE`:

To remove data.

Example:

`DELETE FROM Students WHERE ID = 1;`

3. Data Query Language (DQL):

Used to query and fetch data from the database.

Command:

`SELECT`:

To retrieve data.

Example:

`SELECT * FROM Students WHERE Age > 18;`

4. Data Control Language (DCL):

Used to control user access and permissions.

Commands:

`GRANT`:

To provide access rights.

Example:

`GRANT SELECT ON Students TO User1;`

`REVOKE`:

To remove access rights.

Example:

`REVOKE SELECT ON Students FROM User1;`

5. Transaction Control Commands (TCL):

Used to manage transactions, ensuring consistency and reliability.

Commands:

`COMMIT`:

Saves changes permanently.

Example:

`COMMIT;`

`ROLLBACK`:

Reverts changes made during a transaction.

Example:

`ROLLBACK;`

`SAVEPOINT`:

Creates a point in a transaction to rollback to later.

Example:

`SAVEPOINT SP1;`

Summary:

DDL manages structure, **DML** manipulates data, **DQL** retrieves data, **DCL** controls access, and **TCL** ensures transaction reliability.

These commands collectively provide complete functionality for managing relational databases.

2.3 Data types in SQL:

SQL data types define the type of data that can be stored in a table's columns. They ensure proper storage, retrieval, and processing of data in a database.

1. Numeric Data Types:

Used for storing numeric values, such as integers and decimals.

INT / INTEGER:

Stores whole numbers. Example: `42`

SMALLINT:

Stores smaller integer values.

BIGINT:

Stores very large integers.

DECIMAL / NUMERIC:

Stores fixed-point numbers with defined precision and scale. Example: `DECIMAL(10, 2)` for values like `1234.56`

FLOAT / REAL:

Stores floating-point numbers for approximate values.

2. Character/String Data Types:

Used for storing text or alphanumeric data.

CHAR(n):

Stores fixed-length strings. Example: `CHAR(5)` for values like `"Ali  "`

VARCHAR(n):

Stores variable-length strings. Example: `VARCHAR(50)` for names.

TEXT:

Stores large amounts of text data.

3. Date and Time Data Types:

Used for storing date and time values.

DATE:

Stores dates in the format `YYYY-MM-DD`. Example: `2024-12-31`

TIME:

Stores time in the format `HH:MM:SS`. Example: `14:30:00`

DATETIME / TIMESTAMP:

Stores both date and time. Example: `2024-12-31 14:30:00`

YEAR:

Stores year values. Example: `2024`

4. Binary Data Types:

Used for storing binary data such as images, files, or multimedia.

BLOB (Binary Large Object):

Stores binary data like images or audio.

VARBINARY(n):

Stores variable-length binary data.

5. Boolean Data Type:

BOOLEAN:

Stores true or false values (1 for TRUE, 0 for FALSE).

6. Other Specialized Data Types:

ENUM:

Stores a predefined list of values. Example: `ENUM('Small', 'Medium', 'Large')`

SET:

Stores a set of predefined values, allowing multiple selections.

Summary:

SQL offers a wide range of data types to handle numeric, text, date/time, and binary data efficiently. Choosing the right data type ensures optimized storage, better performance, and data accuracy.

2.4 Creating Tables, Selecting from tables, WHERE Clause:

1. Creating Tables in SQL:

The `CREATE TABLE` statement is used to define the structure of a table, including its columns and their data types.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
);
```

Example:

```
CREATE TABLE Students (
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT,
    Grade CHAR(1)
);
```

This creates a table named `Students` with four columns: `ID`, `Name`, `Age`, and `Grade`.

2. Selecting from Tables in SQL:

The `SELECT` statement is used to retrieve data from a table.

Syntax:

```
SELECT column1, column2, ... FROM table_name;
```

Example:

```
SELECT Name, Age FROM Students;
```

This retrieves the `Name` and `Age` columns for all records in the `Students` table.

To select all columns, use `*`:

```
SELECT * FROM Students;
```

3. WHERE Clause in SQL:

The `WHERE` clause is used to filter records based on specific conditions.

Syntax:

```
SELECT column1, column2, ... FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM Students WHERE Age > 18;
```

This retrieves all columns for students whose `Age` is greater than 18.

Example with multiple conditions:

```
SELECT Name FROM Students WHERE Grade = 'A' AND Age < 20;
```

This retrieves the names of students with a grade of 'A' and age less than 20.

Operators in WHERE Clause:

Comparison Operators:

`=`, `>`, `<`, `>=`, `<=`, `<>` (not equal).

Logical Operators:

`AND`, `OR`, `NOT`.

Pattern Matching:

`LIKE`. Example: `WHERE Name LIKE 'A%'` (names starting with 'A').

Range Checking:

`BETWEEN`. Example: `WHERE Age BETWEEN 18 AND 25`.

Summary:

Creating Tables:

Defines the structure of a database table.

Selecting from Tables:

Retrieves data using the `SELECT` statement.

WHERE Clause:

Filters data based on specified conditions, enabling precise queries.

2.5 Selecting from tables, DISTINCT Clause, Column aliasing:

1. Selecting from Tables in SQL:

The `SELECT` statement is used to retrieve data from one or more tables.

Syntax:

SELECT column1, column2, ... FROM table_name;

Example:

SELECT Name, Age FROM Students;

This retrieves the `Name` and `Age` columns from the `Students` table.

Selecting All Columns:

SELECT * FROM Students;

This retrieves all columns from the table.

2. DISTINCT Clause in SQL:

The `DISTINCT` clause is used to eliminate duplicate rows in the results.

Syntax:

SELECT DISTINCT column1, column2, ... FROM table_name;

Example:

SELECT DISTINCT Grade FROM Students;

This retrieves unique values in the `Grade` column.

Multiple Columns with DISTINCT:

SELECT DISTINCT Age, Grade FROM Students;

This retrieves unique combinations of `Age` and `Grade`.

3. Column Aliasing in SQL:

Column aliasing renames a column in the result set to make it more readable or meaningful.

Syntax:

SELECT column_name AS alias_name FROM table_name;

Example:

SELECT Name AS StudentName, Age AS StudentAge FROM Students;

This displays `Name` as `StudentName` and `Age` as `StudentAge` in the output.

Without AS (Optional):

SELECT Name "Student Name", Age "Student Age" FROM Students;

This produces the same result as above.

Summary:

Selecting from Tables:

Retrieves specific or all columns from a table.

DISTINCT Clause:

Ensures unique results by removing duplicate rows.

Column Aliasing:

Renames columns in the result set for better readability or clarity.

2.6 Manipulation Table data:

Manipulating table data involves **inserting**, **updating**, and **deleting** records within a database table. These operations are performed using **Data Manipulation Language (DML)** commands.

1. Inserting Data into a Table:

The `INSERT` statement adds new records to a table.

Syntax:

INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

Example:

INSERT INTO Students (ID, Name, Age, Grade) VALUES (1, 'Ali', 20, 'A');

This inserts a record with ID `1`, Name `Ali`, Age `20`, and Grade `A` into the `Students` table.

Inserting into All Columns:

INSERT INTO Students VALUES (2, 'Ahmed', 22, 'B');

2. Updating Data in a Table:

The `UPDATE` statement modifies existing records in a table.

Syntax:

UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;

Example:

UPDATE Students SET Age = 21, Grade = 'A+' WHERE ID = 1;

This updates the `Age` to `21` and `Grade` to `A+` for the record where `ID` is `1`.

Note:

Always use the `WHERE` clause to specify which records to update; otherwise, all records will be modified.

3. Deleting Data from a Table:

The `DELETE` statement removes records from a table.

Syntax:

DELETE FROM table_name WHERE condition;

Example:

DELETE FROM Students WHERE ID = 1;

This deletes the record with `ID = 1` from the `Students` table.

Deleting All Records:

DELETE FROM Students;

This deletes all records but keeps the table structure.

4. Truncating Data from a Table:

The `TRUNCATE` command removes all records from a table but resets identity columns.

Syntax:

TRUNCATE TABLE table_name;

Example:

TRUNCATE TABLE Students;

Summary:

INSERT adds new records.

UPDATE modifies existing records.

DELETE removes specific records based on a condition.

TRUNCATE clears all data while preserving the table structure.

These commands enable dynamic manipulation of data within SQL tables.

2.7 Altering Table structure:

The `ALTER TABLE` statement is used to modify the structure of an existing table. This includes adding, modifying, or deleting columns and constraints.

1. Adding a Column:

You can add a new column to an existing table.

Syntax:

ALTER TABLE table_name ADD column_name datatype constraints;

Example:

ALTER TABLE Students ADD Email VARCHAR(50);

This adds a new column `Email` to the `Students` table.

2. Modifying a Column:

You can change the data type, size, or constraints of an existing column.

Syntax:

ALTER TABLE table_name MODIFY column_name new_datatype constraints;

Example:

ALTER TABLE Students MODIFY Age INT NOT NULL;

This modifies the `Age` column to make it a `NOT NULL` column.

For databases like PostgreSQL:

ALTER TABLE Students ALTER COLUMN Age TYPE BIGINT;

3. Renaming a Column:

You can rename an existing column in the table.

Syntax (varies by database)**:**

ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

Example:

ALTER TABLE Students RENAME COLUMN Email TO ContactEmail;

4. Dropping a Column:

You can remove an existing column from the table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Example:

ALTER TABLE Students DROP COLUMN Grade;

This deletes the `Grade` column from the `Students` table.

5. Adding or Dropping Constraints:

You can add or remove constraints (like primary key, foreign key, or unique constraints).

Adding a Constraint:

ALTER TABLE Students ADD CONSTRAINT pk_students PRIMARY KEY (ID);

Dropping a Constraint:

ALTER TABLE Students DROP CONSTRAINT pk_students;

Summary:

The `ALTER TABLE` statement is a powerful tool for modifying table structures dynamically. It allows you to:

Add new columns.

Modify existing columns.

Rename columns.

Drop columns or constraints.

These features help maintain the database structure as requirements evolve.

2.8 Data Constraints: Unique, Not Null, Primary Key, Foreign Key, Check, Default Constraint:

Data constraints are rules applied to table columns to ensure the validity, accuracy, and integrity of the data. Below are the most commonly used constraints:

1. UNIQUE Constraint:

Ensures that all values in a column are unique (no duplicates).

A table can have multiple `UNIQUE` constraints.

Syntax:

```
CREATE TABLE Students (
    ID INT UNIQUE,
```

```
        Name VARCHAR(50)
    );
```

Example:

```
    ALTER TABLE Students ADD CONSTRAINT Unique_Email UNIQUE (Email);
```

2. NOT NULL Constraint:

Ensures that a column cannot have `NULL` values (i.e., the field must always have a value).

Syntax:

```
    CREATE TABLE Students (
        ID INT NOT NULL,
        Name VARCHAR(50) NOT NULL
    );
```

Example:

```
    ALTER TABLE Students MODIFY Age INT NOT NULL;
```

3. PRIMARY KEY Constraint:

Combines `UNIQUE` and `NOT NULL`. Ensures each row has a unique identifier.
A table can only have one `PRIMARY KEY`.

Syntax:

```
    CREATE TABLE Students (
        ID INT PRIMARY KEY,
        Name VARCHAR(50)
    );
```

Example:

```
    ALTER TABLE Students ADD CONSTRAINT PK_Students PRIMARY KEY (ID);
```

4. FOREIGN KEY Constraint:

Establishes a relationship between two tables by referencing the `PRIMARY KEY` of another table.
Ensures referential integrity.

Syntax:

```
    CREATE TABLE Enrollments (
        StudentID INT,
        CourseID INT,
        FOREIGN KEY (StudentID) REFERENCES Students(ID)
    );
```

Example:

```
    ALTER TABLE Enrollments ADD CONSTRAINT FK_Student FOREIGN KEY (StudentID)
  REFERENCES Students(ID);
```

5. CHECK Constraint:

Ensures that all values in a column meet a specific condition.

Syntax:

```
CREATE TABLE Students (
    ID INT,
    Age INT CHECK (Age >= 18)
);
```

Example:

```
ALTER TABLE Students ADD CONSTRAINT Check_Age CHECK (Age BETWEEN 18 AND 25);
```

6. DEFAULT Constraint:

Assigns a default value to a column when no value is provided.

Syntax:

```
CREATE TABLE Students (
    ID INT,
    Grade CHAR(1) DEFAULT 'C'
);
```

Example:

```
ALTER TABLE Students ADD CONSTRAINT Default_Grade DEFAULT 'C' FOR Grade;
```

Summary:

UNIQUE:

No duplicates in a column.

NOT NULL:

Column must have a value.

PRIMARY KEY:

Uniquely identifies each row.

FOREIGN KEY:

Links two tables.

CHECK:

Validates data against a condition.

DEFAULT:

Assigns a default value when no input is given.

These constraints ensure data integrity and help maintain consistent and accurate databases.